

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 1: Introduction, Efficiency, and Stable Matching

Foreign Trade University

Contents

1	Introduction	1
2	What Is an Algorithm, and Why Analyze It?	1
2.1	Measuring running time	1
2.2	Asymptotic notation	2
3	Data Structures Refresher	2
3.1	Arrays vs. linked lists	2
3.2	Stacks and queues	3
3.3	Binary heaps	3
4	Recursion and a First Taste of Divide and Conquer	3
4.1	Naive vs. memoized recursion: Fibonacci	3
4.2	Karatsuba multiplication	4
5	The Stable Matching Problem	4
5.1	Motivation	4
5.2	Formal definition	4
5.3	The Gale–Shapley algorithm	4
5.4	Termination	5
5.5	Correctness: the result is a perfect matching	5
5.6	Correctness: the result is stable	5
5.7	Optimality	6
5.8	Extensions	6
6	Summary and Looking Ahead	6

1 Introduction

Welcome to *Design and Analysis of Algorithms*. This course is built around a simple question: **given a computational problem, how do we design a procedure (an *algorithm*) that solves it correctly, and how do we argue that the procedure is efficient?**

We will follow Kleinberg and Tardos, *Algorithm Design* (2005), and this first week sets up two things that the rest of the course depends on:

1. **A vocabulary for efficiency** – asymptotic (Big-O) notation, and a quick refresher on the basic data structures (arrays, linked lists, stacks, queues, heaps) that we will use as building blocks throughout the semester.
2. **A first complete case study** – the *Stable Matching Problem* and the Gale–Shapley algorithm, which already illustrates the full arc of an algorithms course: a precise problem statement, an algorithm, a proof that it terminates, a proof that it is correct, and an analysis of its running time.

2 What Is an Algorithm, and Why Analyze It?

An **algorithm** is an unambiguous, step-by-step procedure that takes an input satisfying some precondition and produces an output satisfying some desired postcondition, using a finite amount of work.

When we say an algorithm is “good”, we usually mean three things:

- **Correctness**: it always produces the right answer (or we can precisely characterize when it does).
- **Efficiency**: it uses an acceptable amount of time and memory, even as the input grows large.
- **Clarity / implementability**: it can actually be turned into working code.

2.1 Measuring running time

We measure running time as a function of **input size** n , counting the number of *basic operations* (comparisons, arithmetic operations, array accesses, etc.) in the *worst case* over all inputs of size n . We care about *growth rate*, not the exact constant, because:

- Constants depend on hardware, language, and implementation details that change over time.
- For large n , the growth rate is what determines whether an algorithm is usable at all.

2.2 Asymptotic notation

Definition 2.1 (Big-O, Big-Omega, Big-Theta). *Let $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$.*

- $f(n) = O(g(n))$ *if there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. (f grows no faster than g .)*
- $f(n) = \Omega(g(n))$ *if there exist constants $c > 0$ and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$. (f grows at least as fast as g .)*
- $f(n) = \Theta(g(n))$ *if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$. (f and g grow at the same rate.)*

A useful ordering of common growth rates, from slowest to fastest growing:

$$O(1) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!).$$

Why this matters for this course. Throughout the semester, the central question for every algorithm we design will be: *what is the tightest $\Theta(\cdot)$ bound we can prove on its running time, and is that good enough for the problem at hand?* An algorithm that is correct but runs in $O(2^n)$ is often useless in practice once n exceeds a few dozen, while an $O(n \log n)$ algorithm can handle inputs with millions of elements.

Growth rate	Name	Typical example
$O(1)$	constant	array index access
$O(\log n)$	logarithmic	binary search
$O(n)$	linear	scanning a list
$O(n \log n)$	linearithmic	merge sort, heapsort
$O(n^2)$	quadratic	insertion sort, naive multiplication
$O(2^n)$	exponential	brute-force subset enumeration

Table 1: Common running-time classes.

3 Data Structures Refresher

Many algorithms in this course are described at a high level (“find the smallest element”, “process items in FIFO order”, “add to the front of a list”), but their running time depends entirely on *which data structure* is used to store the data. This section reviews four data structures that you will use repeatedly: arrays, linked lists, stacks/queues, and binary heaps.

3.1 Arrays vs. linked lists

An **array** stores elements in contiguous memory. A **linked list** stores elements in nodes, each holding a value and a pointer/reference to the next node.

Operation	Array	Singly linked list
Access element at index i	$O(1)$	$O(n)$
Search for a value	$O(n)$	$O(n)$
Insert/remove at the <i>front</i>	$O(n)$	$O(1)$
Insert/remove at the <i>end</i>	$O(1)$ amortized	$O(1)$ if tail pointer kept, else $O(n)$

Table 2: Cost comparison: arrays vs. linked lists.

The key trade-off: arrays give fast *random access* but expensive insertion/removal at arbitrary positions (because elements must be shifted); linked lists give fast insertion/removal at known positions but no random access. Problem 9 in this week’s practical exercises asks you to measure this difference directly by counting operations.

3.2 Stacks and queues

A **stack** is a Last-In-First-Out (LIFO) container with operations **push** (add to top) and **pop** (remove from top), both $O(1)$. Stacks are the natural structure for nested/recursive structure – e.g., matching brackets, function call frames, undo history (Problem 10).

A **queue** is a First-In-First-Out (FIFO) container with operations **enqueue** (add to back) and **dequeue** (remove from front), both $O(1)$ with a proper implementation (e.g., a circular buffer or a doubly linked list, or in Python `collections.deque`). Queues model “processing in arrival order” – e.g., round-robin CPU scheduling (Problem 11) and breadth-first search (which we will see in a later week).

3.3 Binary heaps

A **binary min-heap** is a complete binary tree (usually stored implicitly in an array) satisfying the *heap property*: every node's key is $\leq$ the keys of its children. This gives:

- **find-min**: $O(1)$ (the root).
- **insert**: $O(\log n)$ (append, then “sift up”).
- **extract-min**: $O(\log n)$ (swap root with last element, remove last, “sift down”).

Repeatedly inserting n elements and extracting the minimum gives **heapsort**, an $O(n \log n)$ sorting algorithm (Problem 16). Heaps will reappear later in the course as the priority queue underlying Dijkstra's shortest-path algorithm and Prim's minimum spanning tree algorithm.

4 Recursion and a First Taste of Divide and Conquer

Many of the algorithms in this course are recursive: they solve a problem by solving one or more smaller instances of the *same* problem and combining the results. We will study divide-and-conquer algorithms formally in a later week, but it is useful to see two examples now, because they reinforce the asymptotic-analysis vocabulary from Section 2.

4.1 Naive vs. memoized recursion: Fibonacci

The naive recursive definition $F(n) = F(n-1) + F(n-2)$ (with $F(0) = 0, F(1) = 1$) makes an exponential number of calls, because the same sub-problems are recomputed many times. If we *memoize* (cache) results, each sub-problem $F(k)$ for $k = 0, \dots, n$ is computed exactly once, giving $O(n)$ total work. Problem 21 asks you to instrument both versions and count calls directly, so you can see the exponential-vs-linear gap with your own eyes.

4.2 Karatsuba multiplication

The grade-school algorithm for multiplying two n -digit numbers performs $\Theta(n^2)$ single-digit multiplications. Karatsuba's algorithm (1960) splits each number into two halves,

$$x = x_1 \cdot 10^{n/2} + x_0, \quad y = y_1 \cdot 10^{n/2} + y_0,$$

and observes that the product $xy = x_1y_1 \cdot 10^n + (x_1y_0 + x_0y_1) \cdot 10^{n/2} + x_0y_0$ can be computed using only **three** recursive multiplications of $n/2$ -digit numbers (instead of the four you would expect), via the identity

$$x_1y_0 + x_0y_1 = (x_1 + x_0)(y_1 + y_0) - x_1y_1 - x_0y_0.$$

Solving the recurrence $T(n) = 3T(n/2) + O(n)$ gives $T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$, asymptotically better than $\Theta(n^2)$. Problem 15 asks you to implement Karatsuba and compare its multiplication count against the naive algorithm.

5 The Stable Matching Problem

We now turn to this week's main case study, which is also the opening example of Kleinberg & Tardos, Chapter 1.

5.1 Motivation

Consider assigning n medical-school graduates (*residents*) to n hospital residency positions. Each resident has a ranked preference list over hospitals, and each hospital has a ranked preference list over residents. We want an assignment with no incentive for any pair to break their assigned match and match with each other instead. The same abstract structure shows up when matching students to schools, organ donors to recipients, or (the classical framing) men to women in a dating-service scenario.

5.2 Formal definition

Definition 5.1 (Stable Matching Problem). *We are given two equal-size sets, M (“men”) and W (“women”), each of size n . Each $m \in M$ has a strict, complete preference list (a total order) over W , and each $w \in W$ has a strict, complete preference list over M . A **perfect matching** S is a set of n pairs (m, w) such that every $m \in M$ and every $w \in W$ appears in exactly one pair.*

*A pair $(m, w) \notin S$ is a **rogue couple** (or blocking pair) for S if*

- *m prefers w to his partner in S , and*
- *w prefers m to her partner in S .*

*S is **stable** if it is a perfect matching with no rogue couples. Otherwise S is **unstable**.*

A naive approach – try all $n!$ perfect matchings and check each for stability – takes factorial time. Problem 5 asks you to implement exactly this brute-force approach for small n , both as a sanity check and to see why we need something smarter.

5.3 The Gale–Shapley algorithm

Algorithm 1 Gale–Shapley (men propose)

```
1: Initialize all  $m \in M$  and  $w \in W$  to be free
2: while some man  $m$  is free and has not proposed to every woman do
3:    $w \leftarrow$  the highest-ranked woman in  $m$ ’s list to whom  $m$  has not yet proposed
4:   if  $w$  is free then
5:      $(m, w)$  become engaged
6:   else if  $w$  prefers  $m$  to her current partner  $m'$  then
7:      $(m, w)$  become engaged;  $m'$  becomes free
8:   else
9:      $w$  rejects  $m$ ;  $m$  remains free
10:  end if
11: end while
12: return the set of engaged pairs
```

Each man proposes down his list in order, never to the same woman twice. Each woman, once engaged, only ever *trades up* – she keeps her current partner unless a more preferred man proposes, in which case she “upgrades” and her old partner becomes free again. Problems 3 and 4 ask you to implement both the men-propose and women-propose versions of this algorithm.

5.4 Termination

Lemma 5.2. *The algorithm terminates after at most n^2 iterations of the while loop.*

Proof. Each iteration corresponds to one man proposing to one woman he has never proposed to before. There are n men, each with at most n women on his list, so there can be at most n^2 proposals total. Since the loop body always consumes one such “not yet proposed” pair, the loop runs at most n^2 times. $\square$

Problem 18 asks you to empirically measure the average number of proposals for random instances and confirm it is well below the n^2 worst-case bound.

5.5 Correctness: the result is a perfect matching

Lemma 5.3. *At termination, every man and every woman is matched (the result is a perfect matching).*

Proof. Once engaged, a woman never becomes free again – she only switches partners, never loses one entirely. So the set of engaged women is monotonically non-decreasing.

Suppose for contradiction that at termination some man m is free. Then m must have been rejected by (or never engaged with) every woman, i.e., m has proposed to all n women and all n women are currently engaged (to other men, since m is free). But there are only n men total, so if all n women are engaged, all n men are engaged – contradicting that m is free. Hence no man is left free, and since $|M| = |W| = n$, every woman is matched too. $\square$

5.6 Correctness: the result is stable

Lemma 5.4. *The matching S returned by Gale–Shapley has no rogue couples.*

Proof. Suppose $(m, w) \notin S$ with m matched to $w' \neq w$ and w matched to $m' \neq m$ in S . We must show (m, w) is *not* a rogue couple, i.e., it is not the case that both m prefers w to w' and w prefers m to m' .

Since men propose in decreasing order of preference, and m ends up matched to w' , one of two things happened regarding w :

- m never proposed to w – but men propose in order of decreasing preference, so this means m prefers w' to w , i.e., m does *not* prefer w to w' .
- m proposed to w at some point and was rejected (either immediately or later displaced). Women only reject/displace a partner in favor of someone they prefer *more*, and a woman’s partner only gets better over time. So w ’s final partner m' is someone w prefers at least as much as m , i.e., w does *not* prefer m to m' .

In either case, (m, w) cannot be a rogue couple. Since (m, w) was an arbitrary pair not in S , S is stable. $\square$

Problems 1 and 2 ask you to implement, from scratch, the perfect-matching check and the rogue-couple/stability check used in these proofs – these are the basic “verifiers” you will reuse for the rest of the week’s exercises.

5.7 Optimality

Definition 5.5. A woman w is a **valid partner** of man m if there exists some stable matching in which m and w are paired. The **best valid partner** of m is the valid partner m ranks highest.

Theorem 5.6 (Men-optimality). In the matching S^* produced by the men-propose Gale–Shapley algorithm, every man is matched with his best valid partner.

Theorem 5.7 (Women-pessimality). In the same matching S^* , every woman is matched with her worst valid partner.

These two theorems together explain a striking asymmetry: *the side that proposes does at least as well as possible (among all stable matchings), and the side that responds does at most as well as possible.* Problems 7 and 8 ask you to verify both theorems computationally by comparing the Gale–Shapley result against the brute-force set of all stable matchings from Problem 5.

5.8 Extensions

Two natural extensions, both explored in this week’s exercises:

- **Incomplete preference lists** (Problem 20): if participants may declare some potential partners “unacceptable”, a stable matching may leave some participants unmatched, but no acceptable pair can both prefer each other to staying unmatched.
- **Hospital–Residents problem** (Problem 19): a many-to-one generalization in which each hospital has a *capacity* > 1 . A resident-proposing algorithm analogous to Gale–Shapley still produces a stable assignment.

6 Summary and Looking Ahead

This week established:

- How to reason about algorithm efficiency using O , Ω , Θ notation, and the standard hierarchy of growth rates.
- The cost trade-offs between arrays, linked lists, stacks, queues, and binary heaps – the building blocks for almost every algorithm in this course.
- A complete worked example (Gale–Shapley) showing the full methodology of this course: *define the problem precisely* $\rightarrow$ *design an algorithm* $\rightarrow$ *prove termination* $\rightarrow$ *prove correctness* $\rightarrow$ *analyze running time* $\rightarrow$ *ask what “optimal” even means.*

Next week, we make the efficiency analysis from Section 2 fully rigorous and look at further worked examples of algorithm analysis (Kleinberg & Tardos, Chapter 2).